GraphRAG for Online History Class Data: A Custom Knowledge Graph and Retrieval Architecture for Domain Specific Audio

Abstract

This document specifies a complete GraphRAG system for transforming ~60 hours of English history class transcriptions into a queryable knowledge graph with LLM-powered retrieval. The system extracts structured knowledge — teacher claims, predictions, opinions, and entity relationships — from class transcripts and stores them in a Neo4j property graph with bi-temporal metadata. A hybrid retrieval layer combines vector similarity search, graph traversal, text-to-Cypher generation, and community-based thematic summaries, exposed to client-side LLMs via an MCP server.

The architecture is custom-built rather than framework-dependent. This decision is grounded in three empirical findings: (1) schema-constrained extraction achieves 98.8% precision versus open extraction's 40% predicate error rate; (2) the LoCoMo benchmark demonstrates that transforming dialogue into structured assertions dramatically outperforms raw transcript retrieval (87.9% human F1 vs. 37.8% for GPT-3.5); and (3) no existing framework — Microsoft GraphRAG, LightRAG, LlamaIndex, or Graphiti — natively supports the intersection of speaker attribution, bi-temporal claim modeling, and Portuguese-language conversational data that this system requires.

Total estimated extraction cost **to be defined**

1. Introduction and Problem Statement

1.1 The Domain

Predictive History is a English speaking history youtube channel with ~1 hour long history classes by prof Jiang. The professor is a highly opinionated individual, giving various opinions and predictions mixed with his factual assertions about history.

1.2 Why a Knowledge Graph

Standard vector-based RAG treats transcripts as flat text chunks, losing three critical dimensions of information:

In history and geopolitics, flat RAG fails because it loses **causality and relationships**. If you ask a flat RAG system, "Why did the Roman Republic fall?", it pulls chunks that mention the fall of Rome. But a Knowledge Graph maps the *Claim* (e.g., "Economic inequality weakened the Senate") directly to the *Person* (Tiberius Gracchus) and the *Event* (Assassination).

Temporal narrative. The professor’s views evolve. Since he makes many predictions about the future his insights might evolve as classes continue.

Structural relationships. The corpus contains implicit networks — , topics that cluster into themes, predictions that can be verified against outcomes. These relationships are invisible to embedding-based retrieval but naturally represented in a graph.

A knowledge graph addresses all three by making speakers, claims, predictions, and their temporal relationships explicit, queryable graph structures.

1.3 Why Custom

The decision to build a custom pipeline rather than adopt an existing framework is justified by converging evidence from three directions.

Framework limitations for conversational data. Microsoft GraphRAG's default extraction recognizes four entity types (Person, Organization, Geography, Event) designed for news articles — none of which capture fine-grained stances,predictions, or opinions. LightRAG performs open extraction with zero schema constraints, producing inconsistent relationship types. Neither handles speaker attribution or claim reification natively. Graphiti/Zep comes closest with its temporal conversation model, but its entity extraction is designed for personal assistant memory, not domain-specific historical dialogue analysis.

Schema-constrained extraction is measurably superior. ODKE+ demonstrated 98.8% precision across 195 predicates with ontology-guided prompting. The Plumber framework analysis found open extraction produces 40% predicate errors. The VLDB 2024 workshop confirmed that ontology conformance markedly increased across all 10 tested domains when extraction was schema-constrained. For a closed domain like History classes, we know the entity types and relationship types in advance — exploiting this knowledge is not optional.

Multi-pass extraction delivers compounding improvements. KGGen (NeurIPS 2025) showed an 18% improvement over GraphRAG via multi-stage extraction. SAC-KG (ACL 2024) achieved 89.32% precision with a Generator→Verifier→Pruner pipeline. ECR demonstrated +16% improvement in triple F1 over single-pass. CORE-KG proved that removing coreference resolution causes a 28.32% increase in node duplication. These gains compound — a custom pipeline can stack all of them.

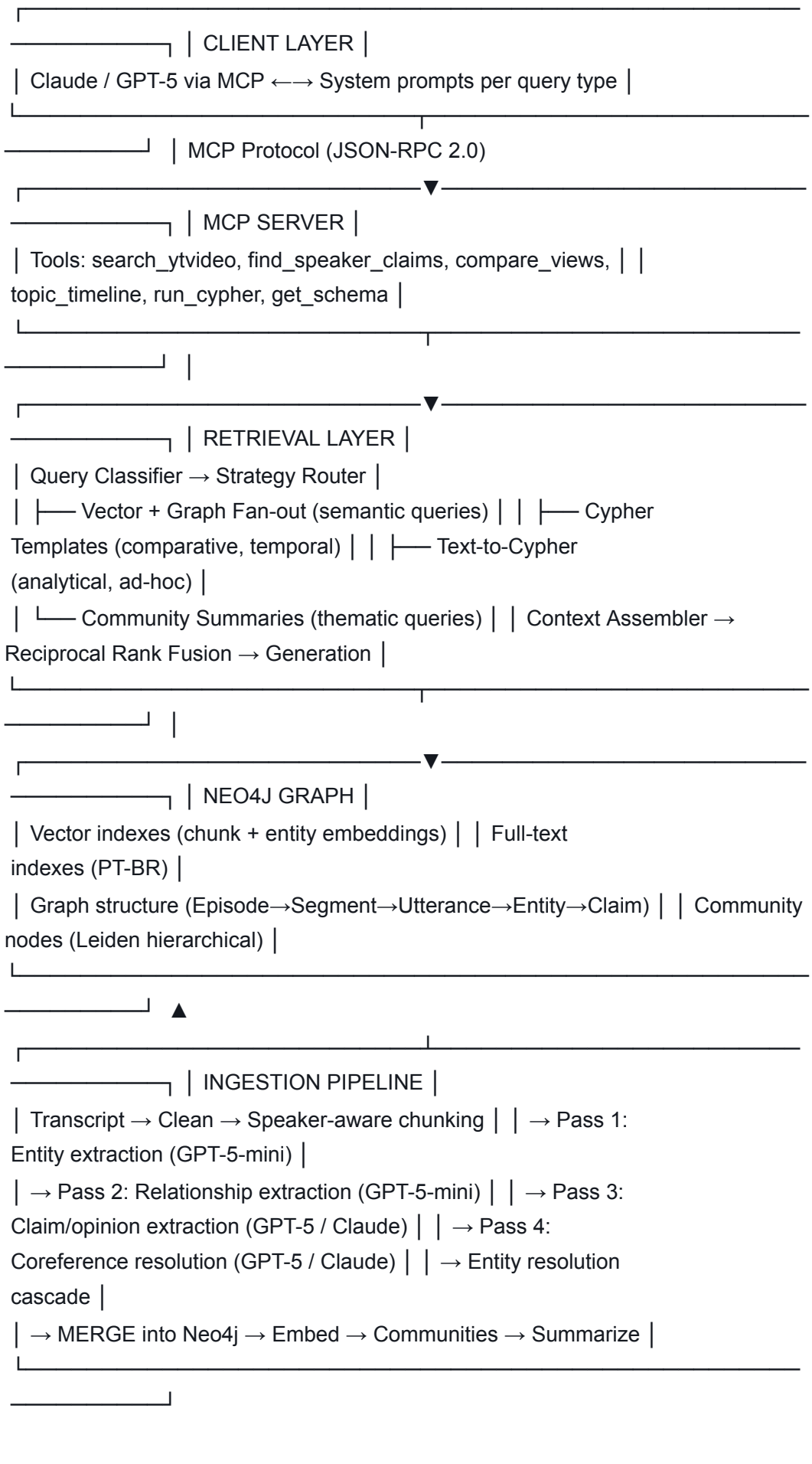
1.4 Target Queries

The system must support four query categories, ordered by complexity:

1. **Factual recall** — "What did prof Jiang say about topic Y?" → entity lookup + vector search
2. **Comparative analysis** — "How do historical figure A and B differ on religion according to Jiang?" → multi entity claim retrieval
3. **Temporal narrative** — "How has predictions on X theory about history evolved throughout the classes?" → chronological claim aggregation
4. **Thematic synthesis** — "What are the dominant themes across all episodes?" → community summaries

Different system prompts at the client layer customize behavior per query type; the retrieval layer must support all four.

2. System Architecture Overview



3. Knowledge Graph Schema

3.1 Design Principles

The schema follows three principles drawn from the research:
Conversation-centric hierarchy. The atomic unit is a speaker utterance, not a document paragraph. The hierarchy is Episode → Segment → Utterance, mirroring Graphiti's Episode subgraph and Di Bratto et al.'s utterance chaining pattern. This preserves the conversational structure that flat chunking destroys.

Claims as first-class nodes (reification). Claims, predictions, and opinions are modeled as intermediate nodes rather than relationship properties. This follows Neo4j's official intermediate node pattern, AIF's I-Node formalism, and Kialo's production validated claim-tree architecture (18,000+ debates). Reification enables attaching arbitrary metadata (confidence, sentiment, outcome status) and modeling claim-to claim relationships (agreement, disagreement, evolution).

Bi-temporal metadata. Every claim and relationship carries two temporal dimensions: when it was said (Episode publish date, via the content hierarchy) and what time period it references (via explicit date properties on claims/predictions). This follows Graphiti's four-timestamp model and TG-RAG's bi-level temporal graph.

3.2 Node Types

The ontology defines seven primary node types across three layers:

Content Layer

Episode — A single class episode.

- Properties: episodeId , title , publishDate (DATE), duration (DURATION), episodeNumber (INT), guestNames (LIST<STRING>), summary (STRING), embedding (LIST<FLOAT>)

Segment — A coherent discussion block within an episode (~10–15 minutes). Segments represent topic-bounded sections — e.g., "Discussion on Alexander the Great Conquests" or "The Emperor’s background and career."

- Properties: segmentId , startTime (DURATION), endTime (DURATION), topic (STRING), summary (STRING), embedding (LIST<FLOAT>)

Utterance — A single speaker turn, preserving the exact text and speaker identity (FOR OUR USE CASE THIS IS SIMPLIFIED, SINCE JIANG IS THE MAIN SPEAKER). This is the chunk unit for vector search.

- Properties: utteranceId , text (STRING), speaker (STRING), turnIndex (INT), tokenCount (INT), embedding (LIST<FLOAT>)

Entity Layer

Person — Historical figures, contemporary leaders, and geopolitical actors discussed in the lectures (e.g., Julius Caesar, Napoleon, current heads of state). • **Properties:** `personId` (STRING), `name` (STRING), `aliases` (LIST<STRING>), `historical_role` (STRING — e.g., ruler, military_leader, philosopher, diplomat), `time_period` (STRING), `description` (STRING), `embedding` (LIST<FLOAT>)

Organization — Companies, regions, empires, religions.

- Properties: orgId , name (STRING), aliases (LIST<STRING>), orgType (STRING — company/fund/bank/regulator/government), description (STRING)

Topic — Religious/Literary themes and economic concepts. Flat hierarchy with category property rather than deep taxonomy.

- Properties: topicId , name (STRING), category (STRING), description (STRING), embedding (LIST<FLOAT>)

Event — Discrete occurrences with dates.

- Properties: eventId , name (STRING), eventType (STRING — policy_decision/ revolutions/election/crisis/data_release), date (DATE), description (STRING)

Analytical Layer

Claim — A reified historical interpretation, geopolitical prediction, or causal assertion made by the speaker (Prof. Jiang). This is the schema's key innovation for moving beyond flat RAG to causal reasoning.

- **Properties:** * `claimId`
  - `text` (STRING)
  - `claimType` (STRING — `factual_assertion` / `historical_interpretation` / `geopolitical_prediction` / `causal_analysis`)
  - **`stance`** (STRING — replacing 'sentiment' — `critical` / `favorable` / `neutral` / `speculative`)
  - `confidence` (FLOAT 0–1 — *represents the LLM's confidence in the extraction*)
  - `status` (STRING — `active` / `superseded` / `verified` / `refuted` / `debated`)
  - `embedding` (LIST<FLOAT>)

Prediction (extends Claim semantically) — A claim with a verifiable future outcome.

- Additional properties: targetDate (DATE), targetMetric (STRING), targetValue (STRING),

outcome (STRING — pending/correct/incorrect/partially_correct), outcomeDate (DATE), outcomeSource (STRING)

Community — A thematic cluster detected by Leiden community detection.

- Properties: communityId , level (INT), name (STRING), summary (STRING), memberCount (INT), embedding (LIST<FLOAT>)

3.3 Relationship Types

Content Hierarchy

- `(:Episode)-[:HAS_SEGMENT {order: INT}]->(:Segment)`
- `(:Segment)-[:HAS_CHUNK {order: INT}]->(:Utterance)` — *Updated from Utterance*
- `(:Segment)-[:NEXT_SEGMENT]->(:Segment)` — sequential chain within episode
- `(:Utterance)-[:NEXT_CHUNK]->(:Utterance)` — sequential chronological chain of the lecture

Historical & Geopolitical Dynamics *(Replaces Speaker Attribution) This layer maps the actual history as described by the lecturer.*

- `(:Person)-[:ROLE_IN {title: STRING, start_date: DATE, end_date: DATE}]->(:Organization)` — maps leadership or affiliation (e.g., Emperor of a Dynasty).
- `(:Person | :Organization)-[:PARTICIPATED_IN {outcome: STRING}]->(:Event)` — connects actors to wars, battles, or treaties.
- `(:Event)-[:LED_TO | :CAUSED]->(:Event)` — crucial for capturing historical timelines and causal chains (e.g., Assassination of Ferdinand `LED_TO` WWI).
- `(:Organization)-[:CONTROLLED | :LOCATED_IN]->(:Topic)` — since Geography is mapped under Topic, this connects polities to territories or trade routes.

Entity Grounding

- `(:Utterance)-[:MENTIONS]->(:Person | :Organization | :Topic | :Event)` — entity co-occurrence and provenance.

Claim Network (The core analytical structure) *Notice that `(:Person)-[:MADE_CLAIM]` is removed, as Jiang is the implicit source of all claims.*

- `(:Claim)-[:EXTRACTED_FROM]->(:Utterance)` — exact provenance to the specific lecture moment.
- `(:Claim)-[:STATED_DURING]->(:Episode)` — temporal anchor to the video release.

- `(:Claim)-[:ABOUT]->(:Person | :Organization | :Topic | :Event)` — what historical actor or concept the claim references.
- `(:Claim)-[:EXPLAINS_CAUSE_OF]->(:Event)` — links Jiang's specific causal analysis to a historical event.
- `(:Claim)-[:SUPPORTS {strength: FLOAT}]->(:Claim)` — agreement or reinforcing theories across different episodes.
- `(:Claim)-[:CONTRADICTS {strength: FLOAT}]->(:Claim)` — nuanced shifts in analysis.
- `(:Claim)-[:REVISES_PREDICTION]->(:Claim)` — *Adapted for "Predictive History"* — links a newer geopolitical prediction that updates or supersedes an older one based on unfolding events.

Community Membership *(Unchanged, great for clustering)*

- `(:Person | :Topic | :Organization)-[:IN_COMMUNITY]->(:Community)`
- `(:Community)-[:PARENT_COMMUNITY]->(:Community)` — Leiden hierarchy

3.4 Graph Constraints and Indexes

cypher

```
-- Uniqueness constraints (prevent duplicate creation)
CREATE CONSTRAINT episode_id IF NOT EXISTS FOR (e:Episode) REQUIRE e.episodeId IS UNIQUE;
CREATE CONSTRAINT segment_id IF NOT EXISTS FOR (s:Segment) REQUIRE s.segmentId IS UNIQUE;
CREATE CONSTRAINT utterance_id IF NOT EXISTS FOR (u:Utterance) REQUIRE u.utteranceId IS UNIQUE;
CREATE CONSTRAINT person_id IF NOT EXISTS FOR (p:Person) REQUIRE p.personId IS UNIQUE;
CREATE CONSTRAINT org_id IF NOT EXISTS FOR (o:Organization) REQUIRE o.orgId IS UNIQUE;
CREATE CONSTRAINT topic_id IF NOT EXISTS FOR (t:Topic) REQUIRE t.topicId IS UNIQUE;
CREATE CONSTRAINT event_id IF NOT EXISTS FOR (ev:Event) REQUIRE ev.eventId IS UNIQUE;
CREATE CONSTRAINT claim_id IF NOT EXISTS FOR (c:Claim) REQUIRE c.claimId IS UNIQUE;

-- Vector indexes for semantic search
CREATE VECTOR INDEX utterance_embeddings IF NOT EXISTS
FOR (u:Utterance) ON (u.embedding)
 OPTIONS {indexConfig: {`vector.dimensions`: 1536, `vector.similarity_function`: 'cosine'}};

CREATE VECTOR INDEX claim_embeddings IF NOT EXISTS
FOR (c:Claim) ON (c.embedding)
 OPTIONS {indexConfig: {`vector.dimensions`: 1536, `vector.similarity_function`: 'cosine'}};

-- Entity Vector Indexes
CREATE VECTOR INDEX person_embeddings IF NOT EXISTS
FOR (p:Person) ON (p.embedding)
 OPTIONS {indexConfig: {`vector.dimensions`: 1536, `vector.similarity_function`: 'cosine'}};

CREATE VECTOR INDEX org_embeddings IF NOT EXISTS
```

```
FOR (o:Organization) ON (o.embedding)
 OPTIONS {indexConfig: {`vector.dimensions`: 1536, `vector.similarity_function`: 'cosine'}};

CREATE VECTOR INDEX topic_embeddings IF NOT EXISTS
FOR (t:Topic) ON (t.embedding)
 OPTIONS {indexConfig: {`vector.dimensions`: 1536, `vector.similarity_function`: 'cosine'}};

CREATE VECTOR INDEX event_embeddings IF NOT EXISTS
FOR (ev:Event) ON (ev.embedding)
 OPTIONS {indexConfig: {`vector.dimensions`: 1536, `vector.similarity_function`: 'cosine'}};

-- Full-text indexes for PT-BR keyword search
CREATE FULLTEXT INDEX utterance_text IF NOT EXISTS FOR (u:Utterance) ON EACH [u.text];
CREATE FULLTEXT INDEX claim_text IF NOT EXISTS FOR (c:Claim) ON EACH [c.text];

-- Composite indexes for common query patterns
CREATE INDEX episode_date IF NOT EXISTS FOR (e:Episode) ON (e.publishDate);
CREATE INDEX claim_type IF NOT EXISTS FOR (c:Claim) ON (c.claimType);
CREATE INDEX claim_status IF NOT EXISTS FOR (c:Claim) ON (c.status);
CREATE INDEX claim_stance IF NOT EXISTS FOR (c:Claim) ON (c.stance);
CREATE INDEX topic_category IF NOT EXISTS FOR (t:Topic) ON (t.category);
```

3.5 Expected Scale

For 134 episodes: ~134 Episode nodes, ~1,300 Segments, ~30000 Utterances, 1 Persons (Jiang), ~100 Organizations, ~100–200 Topics, ~3,000–8,000 Claims, and ~80– 250000 relationships. This is comfortably within Neo4j Community Edition capacity.

4. Ingestion Pipeline

4.1 Preprocessing

Input: Raw transcription output from audio → text (GPT-5-mini via OpenRouter OR gemini API). Transcriptions include speaker labels and timestamps.

Cleaning steps:

1. Normalize EN text — accent normalization, consistent encoding 2. Remove filler words and transcription artifacts (uh, uhm, [inaudible]) 3. Standardize speaker labels — map informal names to canonical forms ("Alexander" → "Alexander the Great")
4. Validate speaker diarization — flag segments where speaker attribution seems inconsistent (Sometimes dr Jiang gets asked nearly inaudible questions, we must detect this and clean it)

4.2 Segment-Aware Chunking

Standard fixed-size chunking destroys segment boundaries. We do not want to cut a chunk when the orator is in the middle of conveying an idea. The chunking strategy must employ a two step chunking logic. One step to best try and determine where an explanation starts and ends so that we can appropriately extract semantic value for the next step of chunking where we in fact chunk the information for proper extraction. So LEVEL 1 CHUNK refers to the first pass LEVEL 2 CHUNK refers to the proper 1500 token max chunk.

Below is a function used for some inspiration on what we want to achieve. The function below is part of a project where we use a chunking strategy that groups complete speaker turns into chunks of ~1,500 tokens with 2-turn overlap, ensuring every chunk preserves who is speaking. Since we are not dealing with speaker turns in the Jiang project, its just food for thought.

python

```
def chunk_by_speaker_turns(
 transcript: list[dict], # [{speaker, text, timestamp}, ...]
max_tokens: int = 1500,
 overlap_turns: int = 2
) -> list[dict]:
 chunks, i = [], 0
    while i < len(transcript):
 chunk_turns, token_count, j = [], 0, i
        while j < len(transcript) and token_count < max_tokens:  turn_tokens =
estimate_tokens(transcript[j]["text"]) if token_count + turn_tokens >
max_tokens and chunk_turns: break
 chunk_turns.append(transcript[j])
 token_count += turn_tokens
 j += 1
 chunks.append({
            "text": "\n\n".join(
                f"[{t['speaker']}]: {t['text']}" for t in chunk_turns
            ),
            "speakers": list(set(t["speaker"] for t in chunk_turns)),
            "start_turn": i,
            "end_turn": j - 1,
            "token_count": token_count
        })
 i = max(j - overlap_turns, i + 1)
    return chunks
```

Rationale: Rook (ISWC 2025) demonstrates that subject-object span distances in dialogue-based relation extraction are higher than in documents — facts distribute across multiple turns. The 2-turn overlap ensures cross-turn assertions are captured in at least one chunk. The 1,500-token window balances extraction quality (larger context) against cost (fewer tokens per call).

Segment detection: Segments (~10–15 minutes of coherent discussion) are detected by topic shift. The simplest approach: embed each chunk, compute cosine similarity between consecutive chunks, and place segment boundaries where similarity drops below a threshold. An LLM-based approach (passing 3–4 chunks and asking "is this still the same topic?") is more accurate but costlier — recommended for v2.

4.3 Multi-Pass Extraction

The pipeline uses five sequential passes with tiered model selection (we

assume five passes using the LEVEL 2 CHUNK, in total it might be more

passes to retrieve the LEVEL 1 CHUNK): **Pass Task Model Temperature Why**

1 Entity extraction GPT-5-mini 0.0 High volume, structured output, cheap

understanding

2 Relationship extraction

GPT-5 or Claude 0.0 Deep contextual reasoning

3 Claim/opinion extraction

4 Coreference resolution

5 Entity resolution Embeddings + GPT-5-mini

GPT-5-mini 0.0 Same chunks, entities as context

0.0 Only ~5% of pairs need LLM

GPT-5 or Claude 0.0 Nuanced stance/sentiment

All extraction passes use **OpenAI Structured Outputs** (or equivalent constrained decoding) to guarantee 100% JSON schema compliance. Define Pydantic models for each pass output and pass them via response_format .

Pass 1: Entity Extraction

Input: Level 2 chunk + episode context (known subjects for this episode) **Output:** List of entities with type, name, aliases, description, speaker attribution

Prompt structure (abbreviated):

```
You are an expert historian and geopolitics analyst extracting entities from a YouTube history class transcript
("Predictive History" by Prof. Jiang).

ENTITY TYPES: person, organization, topic, event, claim
EPISODE CONTEXT: {known entities/subjects/focus for this episode}

RULES:
- Use universally recognized canonical names (e.g., "Alexander the Great", "Roman Empire").
- Include ALL aliases, titles, and abbreviations.
- Attribute every entity and claim to the speaker who mentioned it.
- For PERSON, classify role: (ruler / military_leader / philosopher / diplomat / civilian).
```

- For ORGANIZATION, treat historical polities as organizations. Classify type: (empire / nation-state / dynasty / alliance / religion / institution).

- For TOPIC, assign category: (geography / ideology / economic_system / governing_system / military_strategy / religious_practice).

- For EVENT, classify type: (war / battle / treaty / revolution / crisis / period).

- For CLAIM, extract specific historical interpretations, geopolitical predictions, or analytical assertions made by the speaker (e.g., "The collapse of the Bronze Age was driven by supply chain failures").

TRANSCRIPT CHUNK:

{chunk_text}

Pass 2: Relationship Extraction

Input: Same chunk + entities extracted in Pass 1 **Output:** List of relationships with source, target, type, description, confidence

The entities from Pass 1 are injected as context so the model focuses on relationships between known entities rather than re-extracting them.

Pass 3: Claim/Opinion Extraction

Input: Same chunk + entities from Pass 1 **Output:** List of claims with speaker, text, type, sentiment, referenced entities, time reference **Model:** GPT-5 or Claude (required for nuanced stance detection in English language)

This is the most critical pass. The prompt must distinguish:

- **Factual claims** — "Athens lost the Peloponese War"
- **Opinions** — "I think the USA is a dying empire"
- **Predictions** — "The United States will have a civil war"
- **Recommendations** — "You should read Dante"

Sentiment labels are domain-specific: hawkish/dovish for monetary policy, bullish/ bearish for markets, positive/negative for general economic outlook. (**ADAPT SENTIMENT TO HISTORICAL ANALYSIS)**

Pass 4: Coreference Resolution

Input: Entities from Pass 1 across a sliding window of 3–5 chunks **Output:** Coreference chains mapping pronouns and references to canonical entities

This pass resolves "him" → "Abraham", "the city-state" → "Athens", "the religion" → specific religion being discussed. Critical for English where pronoun usage is frequent and referent ambiguity is high in conversational speech.

4.4 Entity Resolution Cascade

After extraction across all chunks of an episode, entities must be deduplicated. The cascade routes from cheap to expensive:

1. **Exact match** (~40% of duplicates) — Normalize case, whitespace, accents. "united states" = "United States" = "UNITED STATES"
2. **Fuzzy string matching** (~35%) — RapidFuzz token_sort_ratio > 85 on same-type entities. Catches "Phillip the Second" ≈ "Phillip II"
3. **Embedding similarity** (~20%) — Cosine similarity > 0.85 between entity name+description embeddings. Catches "USA" ≈ "United States" when descriptions overlap
4. **LLM judgment** (~5%) — Only truly ambiguous pairs go to GPT-5-mini with both entity contexts for a merge/no-merge decision

Graph-aware deduplication: After initial resolution, run a second pass checking for entities that share >3 relationships with the same targets — these are likely duplicates that string/embedding methods missed.

4.5 Graph Upsert Pattern

All writes use idempotent MERGE patterns. The critical Neo4j rule: never MERGE entire patterns with unbound variables. Always bind nodes first, then MERGE relationships between bound nodes.

```cypher
-- Entity upsert with description enrichment
UNWIND $entities AS entity
MERGE (e:Entity {id: entity.id})
ON CREATE SET e.name = entity.name, e.type = entity.type,
 e.description = entity.description, e.sourceCount = 1
ON MATCH SET e.description = CASE
    WHEN size(e.description) < size(entity.description)
    THEN entity.description ELSE e.description END,
 e.sourceCount = e.sourceCount + 1

-- Relationship upsert between bound nodes
UNWIND $relationships AS rel
MATCH (source {id: rel.sourceId})
MATCH (target {id: rel.targetId})
MERGE (source)-[r:RELATED_TO {type: rel.type}]->(target)
ON CREATE SET r.description = rel.description, r.weight = 1
ON MATCH SET r.weight = r.weight + 1
```

4.6 Embedding Generation

After all entities and claims are in the graph, generate embeddings using `text-embedding-3-small` (1536 dimensions, cheap, fast):

- **Utterance embeddings:** Full utterance text

- **Claim embeddings:** Claim text + speaker name + topic
- **Entity embeddings:** Entity name + description + aliases

4.7 Community Detection

Run Leiden community detection via Neo4j GDS on the entity co-occurrence graph:

```cypher
-- Project entity co-occurrence graph
CALL gds.graph.project(
  'ytvideo-kg',
  ['Person', 'Organization', 'Topic'],
  {
 ABOUT: {orientation: 'UNDIRECTED'},
 AFFILIATED_WITH: {orientation: 'UNDIRECTED'},
 MADE_CLAIM: {orientation: 'UNDIRECTED'}
  }
)

-- Run Leiden with hierarchical output
CALL gds.leiden.write('ytvideo-kg', {
 writeProperty: 'communityId',
 includeIntermediateCommunities: true,
 randomSeed: 42
})
YIELD communityCount, modularity, ranLevels
```

For each detected community, gather member entities and generate a thematic summary via LLM:

```python
def summarize_community(community_id: int, level: int) -> str:
 members = session.run("""
 MATCH (e) WHERE e.communityId = $cid
 OPTIONAL MATCH (e)-[r]-(other) WHERE other.communityId = $cid  RETURN
e.name, labels(e), e.description, type(r), other.name
 LIMIT 100
 """, cid=community_id)

 context = format_community_members(members)
 summary = llm.invoke(f"""
 Summarize this community of related entities from a Brazilian
 investment ytvideo. Include: key themes, notable claims,
 important speakers, and why these entities cluster together.
    {context}
 """)
    return summary
```

Store summaries as Community nodes with embeddings for vector retrieval.

4.8 Cost Estimation

Three mechanisms compound for cost reduction:

Mechanism	Discount	Application
Prompt caching	50% on cached prefix	System prompt + schema identical across chunks
Batch API	50% on all tokens	Historical backlog (24h completion acceptable)
Model tiering	~10× cheaper	GPT-5-mini handles 80% of extraction

Per-episode estimate (1 hours, ~17K tokens):

- Passes 1–2 (GPT-5-mini): ~$0.02
- Pass 3 (GPT-5/Claude): ~$0.04
- Pass 4 (GPT-5/Claude): ~$0.02
- Embeddings: ~$0.01
- Community summarization (amortized): ~$0.01

Total per episode: ~$0.02–0.8 134-episode backlog: ~$7–24 Incremental weekly episode: <$0.8

5. Retrieval Architecture

5.1 Query Classification

A lightweight classifier (GPT-5-mini, zero temperature, structured output) categorizes incoming queries into one of four types, each routed to an optimal retrieval strategy: **(FLAGGED, need to fix this section)**

Query Type	Example	Primary Strategy	Fallback
Factual	"What did X say about Y?"	Vector + graph fan-out	Text-to-Cypher
Comparative	"How do A and B differ on Z?"	Selic evolved?" Cypher template (claims by speaker)	(chronological claims) Vector search per speaker
Temporal	"How has the view on	Cypher template	Vector with date filter

Thematic "What are the main investment themes?"

The classifier prompt:

Community summaries Leiden hierarchy traversal

Classify this query about a History class ytvideo.
Types: factual, comparative, temporal, thematic
Also extract: mentioned_topics, time_range (if any) Query: {user_query}

5.2 Vector + Graph Fan-Out (Factual Queries)

The core retrieval pattern: vector search finds entry-point chunks, then Cypher traversal fans out into the graph neighborhood for structured context. (**THE EXAMPLE BELOW PERTAINS TO A SPEAKER ORIENTED WORK, FOR OURS IT ILL PROBABLY BE SOMETHING SIMILAR BUT DIFFERENT)**

cypher

```
CALL db.index.vector.queryNodes('utterance_embeddings', $topK, $queryEmbedding) YIELD
node AS utterance, score

OPTIONAL MATCH (utterance)<-[:HAS_UTTERANCE]-(seg:Segment)<-[:HAS_SEGMENT]-(ep:Episode OPTIONAL
MATCH (p:Person)-[:SPOKE]->(utterance)
OPTIONAL MATCH (claim:Claim)-[:STATED_IN]->(utterance)
OPTIONAL MATCH (claim)-[:ABOUT]->(topic:Topic)

RETURN utterance.text AS text, score,
 ep.title AS episode, ep.publishDate AS date,
 p.name AS speaker,
      collect(DISTINCT {
 claim: claim.text,
 type: claim.claimType,
 sentiment: claim.sentiment,
 topic: topic.name
      }) AS claims
ORDER BY score DESC
```

When the classifier detects a specific speaker mention, add a speaker filter:

cypher

```
WHERE p.name = $speakerName OR $speakerName IN p.aliases
```

5.3 Cypher Templates (Comparative and Temporal Queries) (THE EXAMPLE BELOW PERTAINS TO A SPEAKER-ATTRIBUTION ORIENTED WORK, FOR OURS IT ILL PROBABLY BE SOMETHING SIMILAR BUT DIFFERENT)

Comparative queries retrieve claims from multiple speakers on the same topic:

```cypher
MATCH (p:Person)-[:MADE_CLAIM]->(c:Claim)-[:ABOUT]->(t:Topic)
WHERE p.name IN $speakers AND toLower(t.name) CONTAINS toLower($topic) MATCH
(c)-[:STATED_DURING]->(ep:Episode)
OPTIONAL MATCH (c)-[stance:SUPPORTS|CONTRADICTS]->(other:Claim)
RETURN p.name AS speaker, c.text AS claim, c.sentiment,
 c.claimType, ep.publishDate AS date,
      collect(DISTINCT {
 relation: type(stance),
 otherClaim: other.text
      }) AS stances
ORDER BY p.name, ep.publishDate
```

Temporal queries aggregate claims chronologically: (**THE EXAMPLE BELOW PERTAINS TO A SPEAKER ORIENTED WORK, FOR OURS IT ILL PROBABLY BE SOMETHING SIMILAR BUT DIFFERENT)**

```cypher
MATCH (c:Claim)-[:ABOUT]->(t:Topic)
WHERE toLower(t.name) CONTAINS toLower($topic)
MATCH (c)-[:STATED_DURING]->(ep:Episode)
MATCH (p:Person)-[:MADE_CLAIM]->(c)
WHERE ep.publishDate >= $startDate AND ep.publishDate <= $endDate
OPTIONAL MATCH (c)-[:EVOLVED_INTO]->(later:Claim)
RETURN ep.publishDate AS date, p.name AS speaker,
 c.text AS claim, c.sentiment, c.claimType,
 later.text AS evolvedTo
ORDER BY date ASC
```

5.4 Text-to-Cypher (Analytical and Ad-Hoc Queries)

For queries that don't fit templates, the system generates Cypher from natural language. The prompt includes the full graph schema, few-shot examples, and strict read-only constraints.

When a generated query fails, the error message and schema are fed back to the LLM for self-correction — up to 2 retries. This "self-healing" pattern is standard practice in Neo4j's Text2Cypher documentation.

Few-shot examples (critical for accuracy):

```
Q: "How many episodes featured discussions about War?"
MATCH (ep:Episode)-[:HAS_SEGMENT]->(s:Segment)-[:HAS_UTTERANCE]->(u:Utterance)
-[:MENTIONS]->(t:Topic {name: 'Selic'})
RETURN count(DISTINCT ep) AS episodeCount

Q: "Which guests made the most predictions?" (FLAGGED, SPEAKER-RELATED WORK)
MATCH (p:Person)-[:MADE_CLAIM]->(c:Claim {claimType: 'prediction'})
```

```
RETURN p.name AS guest, count(c) AS predictions
ORDER BY predictions DESC LIMIT 10
```

Q: "What percentage of Selic predictions were correct?" **(FLAGGED, SPEAKER-RELATED WORK)**

```
MATCH (c:Claim {claimType: 'prediction', status: 'verified'})-[:ABOUT]->(t:Topic) WHERE
toLower(t.name) CONTAINS 'selic'
WITH count(c) AS correct
MATCH (c2:Claim {claimType: 'prediction'})-[:ABOUT]->(t2:Topic)
WHERE toLower(t2.name) CONTAINS 'selic' AND c2.status IN ['verified', 'refuted'] WITH
correct, count(c2) AS total
RETURN correct, total, toFloat(correct) / total AS accuracy
```

5.5 Community Summaries (Thematic Queries)

For "what are the main themes" questions, retrieve community summaries at the appropriate Leiden hierarchy level:

```cypher
MATCH (c:Community)
WHERE c.level = $level
RETURN c.name, c.summary, c.memberCount
ORDER BY c.memberCount DESC
LIMIT $topK
```

For more specific thematic queries, use vector search on community summary embeddings to find the most relevant communities, then retrieve their member entities and claims.

5.6 Context Assembly and Generation

Results from all active strategies are fused using Reciprocal Rank Fusion:

```python
def reciprocal_rank_fusion(
 result_lists: list[list[dict]],
 k: int = 60
) -> list[dict]:
 scores = {}
    for results in result_lists:
        for rank, item in enumerate(results):
 key = item["id"]
 scores[key] = scores.get(key, 0) + 1.0 / (k + rank + 1)
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)
```

The assembled context separates raw text passages from structured graph context:

```python
def assemble_context(vector_results, graph_context, t2c_results=None):  parts =
["## Relevant Passages"]
```

```
    for r in vector_results[:5]:
        parts.append(f"[{r['speaker']}, {r['episode']}, {r['date']}]: {r['text']}")

    parts.append("\n## Knowledge Graph Context")
    for r in graph_context:
        for claim in r.get('claims', []):
            parts.append(
                f"- {claim['topic']}: {claim['claim']} "
                f"({claim['sentiment']}, {claim['type']})"
            )

    if t2c_results:
        parts.append("\n## Structured Query Results")
        for row in t2c_results[:10]:
            parts.append(str(row))

    return "\n".join(parts)
```

6. MCP Server Design

6.1 Tool Interface

The MCP server exposes six tools covering the four query types plus exploration:

```python
from mcp import FastMCP

mcp = FastMCP("market-makers-kg")

@mcp.tool()
async def search_ytvideo(query: str, top_k: int = 5) -> str:
    """Semantic search across ytvideo knowledge graph.
    Returns relevant passages with speaker attribution and related claims."""

@mcp.tool()
async def find_speaker_claims( (FLAGGED, SPEAKER-RELATED WORK, OUR WILL BE SOMETHING DIFFERENT)
    speaker: str, topic: str = None, claim_type: str = None
) -> str:
    """Find all claims made by a specific speaker,
    optionally filtered by topic and claim type."""

@mcp.tool() (FLAGGED, adjust for our use case)
async def compare_views(topic: str, speakers: list[str] = None) -> str: """Compare
different speakers' views on a topic across episodes.  If speakers not specified,
returns all speakers with views on the topic."""
```

```
@mcp.tool()
async def topic_timeline(
 topic: str, start_date: str = None, end_date: str = None
) -> str:
    """Chronological view of how claims about a topic evolved over time."""

@mcp.tool()
async def run_cypher(question: str) -> str:
    """Translate a natural language question into a Cypher query and execute it.  Use for
analytical questions about counts, rankings, and aggregations."""

@mcp.tool()
async def get_schema() -> str:
    """Return the knowledge graph schema — node types, relationship types,  and their
properties. Use to understand what's queryable."""
```

6.2 Design Principles

Following the MCP tool design from our earlier work:

- **Domain tools for common queries** — search_ytvideo , find_speaker_claims , compare_views , topic_timeline are optimized, fast, and easy for the agent to use correctly
- **Generic tools for exploration** — run_cypher and get_schema provide flexibility for unanticipated queries
- **Schema exposed** — get_schema lets the agent discover what's queryable and plan tool calls
- **Explicit limits** — every tool caps results to prevent context overflow • **Structured data returned** — the MCP server returns structured context; the client-side LLM handles natural language generation

7. Alternative Approaches and Trade-Off Analysis

7.1 Framework Comparison

Dimension	Custom Pipeline	Microsoft GraphRAG	LightRAG	Graphiti/Zep	neo4j graphrag
Claim reification	Native	None	None	Four timestamp model	Manual
Bi-temporal modeling	Native	None	None	Via entity edges	Manual
Speaker attribution	Native	None	None	Fact strings on edges	None
PT-BR optimization	Full	English default	agnostic	agnostic	agnostic
Custom prompts	Four entity types	Language	Language	Language	
Schema control	Open	extraction	Generic entities	Configurable	

		$15–45 ~$33K+ (estimated)			
		~$15–30 Varies Varies			
Indexing cost (300 ep)					
	Idempotent MERGE	Manual			
	Partial reindex				
Incremental updates	Graph union Dynamic				
	detection	hierarchical	Optional		
	Leiden via GDS	None Label			
Community	Leiden	propagation			
					~100 Varies Varies
Query tokens ~100–1000 ~610,000 (global)					

7.2 When Each Alternative Would Be Preferable

Microsoft GraphRAG — Choose when: the corpus is static English documents (not conversation), global thematic summarization is the primary query type, and indexing cost is not a concern. Its hierarchical community summaries remain best-in-class for "what are the main themes" queries over large document collections.

LightRAG — Choose when: fast setup is more important than extraction quality, the domain doesn't require speaker attribution or temporal tracking, and incremental updates with minimal engineering are required. Its 6,000× token efficiency over GraphRAG's global search is compelling for cost-sensitive deployments.

LazyGraphRAG — Choose when: you want GraphRAG-quality answers without upfront indexing cost. Its 0.1% indexing cost and 95/96 statistical wins over alternatives make it the default recommendation for document-based (non-conversational) GraphRAG. However, it has no conversation-specific features.
Graphiti/Zep — Choose when: the primary use case is conversational agent memory rather than analytical knowledge extraction. Its bi-temporal model is excellent, but its entity extraction is optimized for personal assistant patterns (tracking user preferences, personal facts) rather than domain-specific financial analysis.

neo4j-graphrag Python package — Choose when: you want pre-built retriever components (VectorRetriever, HybridCypherRetriever, Text2CypherRetriever) without building from scratch. Our system can and should use these retriever implementations within the custom pipeline — they are components, not competitors.

7.3 RDF vs. Property Graph

This system uses Neo4j (property graph) rather than RDF/SPARQL for five reasons:

1. **Schema flexibility** — Property graphs allow rich attributes on both nodes and edges without the blank-node workarounds RDF requires for relationship properties
2. **Query expressiveness** — Cypher's pattern matching is more natural than SPARQL for

multi-hop graph traversal, especially for the comparative and temporal query patterns in Section 5

3. **Ecosystem alignment** — Neo4j GDS provides native Leiden community detection; no equivalent exists for RDF stores
4. **LLM integration** — Neo4j's vector indexes, neo4j-graphrag package, and Text2Cypher tooling are more mature than RDF equivalents
5. **ISO GQL standardization** — The first new ISO database language since SQL (ISO/IEC 39075:2024) eliminates RDF's historical standardization advantage

The trade-off: RDF provides formal ontological reasoning (OWL inference) and interoperability with Linked Data (Wikidata, DBpedia). If the system later needs to integrate with external knowledge bases, an RDF export layer can be added without changing the core architecture.

8. Quality Assurance

8.1 Extraction Validation Queries

Run after each episode ingestion:

cypher

```cypher
-- Orphan entities (no relationships = likely noise)
MATCH (e) WHERE NOT (e)--() AND NOT e:Episode AND NOT e:Community
RETURN labels(e)[0] AS type, count(e) AS orphans;

-- Claims without speaker attribution
MATCH (c:Claim) WHERE NOT (c)<-[:MADE_CLAIM]-(:Person)
RETURN count(c) AS unattributed_claims;

-- Claims without topic grounding
MATCH (c:Claim) WHERE NOT (c)-[:ABOUT]->()
RETURN count(c) AS ungrounded_claims;

-- Duplicate relationships
MATCH (a)-[r1]->(b), (a)-[r2]->(b)
WHERE id(r1) < id(r2) AND type(r1) = type(r2)
RETURN type(r1) AS relType, count(r1) AS duplicates;

-- Entity connectivity distribution (detect super-nodes and isolates)
MATCH (e)
WITH labels(e)[0] AS type, size((e)--()) AS degree
RETURN type, min(degree), avg(degree), max(degree), percentileCont(degree, 0.95);
```

8.2 Confidence Thresholds

- Extraction confidence < 0.7: flagged for review, not ingested

• Entity resolution similarity < 0.8: no automatic merge, logged for manual review • Community summaries regenerated when >20% of member entities change

8.3 Sampling-Based Human Validation

For the initial backlog, sample 5–10 episodes and manually verify:

• Entity extraction recall (are named entities being found?)

• Claim attribution accuracy (are claims attributed to the correct speaker?) • Relationship correctness (do extracted relationships reflect what was said?) • Sentiment labeling accuracy (are positive/negative labels correct?)

Target: >85% accuracy across all dimensions before full backlog processing. **9.**

Project Structure

```
ytvideo-graphrag/
├── src/
│   ├── config.py # Neo4j, OpenRouter, embedding config │   ├──
preprocessing/
│   │   ├── text_cleaner.py # PT-BR normalization, filler removal │   │   └──
chunker.py # Speaker-aware chunking │   ├── extraction/
│   │   ├── prompts.py # All extraction prompts │   │   ├── schemas.py # Pydantic
models for structured output │   │   ├── extractor.py # Multi-pass extraction
orchestrator │   │   ├── claims_extractor.py # Pass 3: claim/opinion extraction │
│   ├── coref_resolver.py # Pass 4: coreference resolution │   │   └──
entity_resolver.py # Entity resolution cascade │   ├── graph/
│   │   ├── neo4j_client.py # Connection, constraints, indexes │   │   ├──
upsert.py # MERGE patterns
│   │   └── communities.py # Leiden + summarization │   ├──
retrieval/
│   │   ├── classifier.py # Query type classification │   │   ├──
vector_search.py # Vector + graph fan-out │   │   ├── cypher_templates.py #
Comparative, temporal queries │   │   ├── text2cypher.py # NL → Cypher
with self-healing │   │   ├── community_search.py # Thematic retrieval │   │
└── assembler.py # Context assembly + RRF │   ├── generation/
│   │   └── generator.py # Final answer generation │   └── mcp/
│   └── server.py # MCP server (6 tools)
├── scripts/
│   ├── ingest_episode.py # Process single new episode │   ├──
full_rebuild.py # Initial backlog processing │   ├── validate_graph.py
```

QA queries
│ └── benchmark_retrieval.py # Retrieval quality testing ├── prompts/ #
Version-controlled prompt templates ├── data/
│ ├── cache/ # LLM response cache (hash → result) │ └── transcripts/
Raw transcription files
└── tests/
├── test_extraction.py
├── test_entity_resolution.py
└── test_retrieval.py

10. Implementation Roadmap

Phase 1: Foundation (Weeks 1–2)

- Set up Neo4j instance with constraints and indexes
- Implement text cleaning and speaker-aware chunking
- Build Pass 1–2 extraction (entities + relationships) with structured outputs • Implement idempotent graph upsert patterns
- Process 5 test episodes, validate extraction quality

Phase 2: Analytical Layer (Weeks 3–4)

- Build Pass 3 (claim/opinion extraction)
- Build Pass 4 (coreference resolution)
- Implement entity resolution cascade
- Add embedding generation pipeline
- Process 20 episodes, validate claim attribution accuracy

Phase 3: Retrieval (Weeks 5–6)

- Implement query classifier
- Build vector + graph fan-out retrieval
- Build Cypher templates for comparative and temporal queries • Implement Text-to-Cypher with self-healing
- Set up community detection and summarization

Phase 4: MCP and Integration (Weeks 7–8)

- Build MCP server with 6 tools
- Integrate with client-side LLM (Claude/GPT-5)

- Build context assembler with RRF
- End-to-end testing across all four query types

Phase 5: Scale (Weeks 9–10)

- Process full 300-episode backlog via Batch API
- Tune community detection (gamma parameter, hierarchy levels) •

  Implement incremental update pipeline for new episodes
- Build monitoring and quality dashboards

11. Key References

Knowledge Graph Schema: Graphiti/Zep (arXiv:2501.13956) · SGMem (arXiv:2509.21212) · MAGMA (arXiv:2601.03236) · Di Bratto et al. (2021) · DialogueGCN (EMNLP 2019) · AIF/DialAM-2024 · Neo4j Intermediate Node Pattern · Joel Chan Discourse Graphs (arXiv:2407.20666)

Extraction: KGGen (NeurIPS 2025) · SAC-KG (ACL 2024) · ODKE+ (98.8% precision) · ECR (+16% triple F1) · CORE-KG (28% duplication reduction) · iText2KG (WISE 2024) · OpenAI Structured Outputs · Text2KGBench-LettrIA (2025)

Retrieval: Neo4j GraphRAG Python package · Text2Cypher 2024 Benchmark · Reciprocal Rank Fusion (Cormack et al., SIGIR 2009) · Adaptive-RAG (NAACL 2024) · LazyGraphRAG (Microsoft Research, 2024)

Conversational Data: LoCoMo (ACL 2024, 87.9% human F1) · Rook OKE-D (ISWC 2025) · DialogRE (ACL 2020) · C3 Benchmark (EMNLP 2025) · "Does Memory Need Graphs?" (arXiv:2601.01280)

Frameworks: Microsoft GraphRAG (arXiv:2404.16130) · LightRAG (HKU, 2024) · GraphRAG-Bench (ICLR 2026) · Leiden Algorithm · DF Leiden (arXiv:2405.11658)